

Clautolisp Development Plan

Codex

June 24, 2026

Contents

1 Purpose	1
2 Current Implementation State	1
3 Primary Architectural Position	2
4 AutoLISP Versus Common Lisp	2
4.1 AutoLISP Should Remain a Distinct Language Layer	2
4.2 Common Lisp Should Provide the Implementation Substrate	3
4.3 Consequence for the Reader	3
5 Proposed Module Breakdown	4
5.1 Phase 1 Modules	4
5.2 Phase 2 Modules	6
5.3 Phase 3 Modules	7
6 Recommended Internal Representation	8
6.1 Reader Result	8
6.2 Symbol Model	9
6.3 Environments	9
7 Dialect Strategy	9
8 Host Environment Profiles	10
9 Directory and System Layout	11
10 Execution Tracking	12

11 Testing Strategy	12
11.1 Core Principles	12
11.2 Test Layers	12
11.3 Test Matrix	13
11.4 Test Assets Needed Early	13
12 Short Decision Summary	13

1 Purpose

This document turns the current specification drafts into an executable development plan.

It focuses on the main architectural question of the project: what belongs to AutoLISP semantics, what belongs to Common Lisp implementation, and which modules are needed to keep that boundary clean.

2 Current Implementation State

The plan below remains architecturally valid, but the repository is no longer in the purely speculative state reflected by the earliest draft.

Current state:

- The `autolisp-reader` module exists and is implemented.
- The reader owns lexing, parsing, source locations, comment handling, and strict versus lax token acceptance.
- Reader-focused tooling exists, including standalone batch executables and a batch wrapper for corpus validation.
- SBCL and CCL test execution, FiveAM integration, and CI/container support are in place.
- The `autolisp-runtime` and `autolisp-builtins-core` modules now exist in initial form.
- Special operators are still future evaluator work and are intentionally kept out of the builtin-function layer.
- The host abstraction and full evaluator remain future work.

3 Primary Architectural Position

The project should implement an AutoLISP system in Common Lisp, not compile AutoLISP away into ad hoc Common Lisp forms and not try to pretend that AutoLISP source is normal Common Lisp source.

That implies three rules:

1. AutoLISP source must be read by a dedicated AutoLISP reader layer.
2. AutoLISP evaluation semantics must be implemented explicitly.
3. Common Lisp should be used as the implementation and extension language, but not as the semantic definition of AutoLISP itself.

This avoids the main long-term risk of the project: leaking Common Lisp semantics into AutoLISP behavior in subtle ways such as package handling, number syntax, symbol case, dynamic scope, error signaling, function/value cells, and reader behavior.

4 AutoLISP Versus Common Lisp

4.1 AutoLISP Should Remain a Distinct Language Layer

The runtime should preserve:

- AutoLISP symbol naming and case rules.
- AutoLISP reader syntax and tokenization.
- Dynamic scoping semantics.
- AutoLISP truthiness and nil handling.
- AutoLISP error behavior, including ***error*** conventions.
- AutoLISP built-in function names and argument conventions.
- Dialect differences between Autodesk AutoLISP, Visual LISP, and BricsCAD compatibility extensions.

4.2 Common Lisp Should Provide the Implementation Substrate

Common Lisp should provide:

- Memory management.
- Data structure implementation.
- Conditions and restarts for internal error handling.
- Modularization with ASDF packages and systems.
- Test harnesses.
- Optional implementation-specific delivery support for SBCL and CCL.

4.3 Consequence for the Reader

A custom readtable with reader macros may help for experimentation or REPL convenience, but it should not be the core parsing strategy.

The safer architecture is:

- A dedicated lexer and parser for AutoLISP source.
- A dialect profile controlling syntax variations.
- An explicit AST or reader-object representation.
- A bridge that can expose parsed forms to a Common Lisp REPL if needed.

Reason:

- Common Lisp readtables are powerful, but they are optimized for Common Lisp reader semantics.
- Good source locations, precise syntax diagnostics, and dialect-dependent parsing are easier with an explicit reader.
- Future support for partial parsing, editor tooling, and source-to-source migration also benefits from a real parser.

Therefore the first module should be a strong **reader** module, but it should be stronger than a mere set of reader macros: it should own lexing, parsing, source locations, and dialect selection.

5 Proposed Module Breakdown

5.1 Phase 1 Modules

5.1.1 `autolisp-reader`

Responsibility:

- Decode input streams from external files.
- Normalize line endings at the stream boundary.
- Detect or accept declared external encodings.
- Tokenize AutoLISP syntax.
- Parse forms into source-aware objects.
- Support dialect switches for AutoCAD, Visual LISP, and BricsCAD differences.

Key outputs:

- Tokens with source positions.
- Parsed forms or AST nodes.
- Syntax errors with accurate file, line, and column reporting.

Why first:

- Everything else depends on trustworthy reading.
- It defines the language boundary.
- It is the most likely place for dialect drift if postponed.

5.1.2 `autolisp-runtime`

Responsibility:

- Define the core runtime object model.
- Represent symbols, environments, function bindings, and dynamic frames.

- Implement evaluation of literals, symbol lookup, function application, special forms, and lambda objects.
- Define truthiness, list behavior, and basic value conversions.

Notes:

- This module should not depend on CAD or UI code.
- It should depend only on `autolisp-reader` and low-level utilities.
- Special operators should be designed and implemented as part of the evaluator in this module layer, not as ordinary builtins.

5.1.3 `autolisp-builtins-core`

Responsibility:

- Numeric functions.
- List functions.
- Equality predicates.
- String and character functions that are part of baseline AutoLISP.
- I/O primitives that do not require CAD integration.

Rationale:

- Builtins need separation from the evaluator so they can be tested independently.
- Some builtins will require dialect flags or compatibility shims.
- Special operators such as `foreach` do not belong here; they require evaluator-controlled argument and body handling.

5.1.4 `autolisp-conditions`

Responsibility:

- Define internal Common Lisp conditions.
- Map them to AutoLISP-visible error behavior.
- Support `*error*` callbacks and controlled recovery paths.

5.1.5 autolisp-host-api

Responsibility:

- Define the abstract host protocol used by the runtime.
- Separate runtime semantics from CAD integration.
- Provide interfaces for entities, selection sets, command execution, system variables, dialogs, and filesystem access where needed.

This module should be an interface layer, not an implementation layer.

5.1.6 autolisp-testkit

Responsibility:

- Test helpers.
- Fixtures.
- Golden-file support.
- Snapshot comparison tools.
- Dialect matrix test runners.

This should appear early, not late, because the reader and runtime both need heavy conformance testing.

5.2 Phase 2 Modules

5.2.1 autolisp-vl

Responsibility:

- Visual LISP compatibility functions.
- File/path utilities.
- String helper families.
- COM-related APIs as stubs, abstractions, or selective implementations depending on host capabilities.

5.2.2 autolisp-dcl-reader

Responsibility:

- Parse DCL files.
- Preserve source locations and dialog structure.

5.2.3 autolisp-dcl-runtime

Responsibility:

- Runtime dialog model.
- Tile state.
- Callback registration and dispatch.

5.2.4 autolisp-mock-host

Responsibility:

- Deterministic host implementation for tests.
- Mock drawing/entity model.
- Stable snapshots for regression testing.

5.3 Phase 3 Modules

5.3.1 autolisp-host-autocad

Responsibility:

- Adapter to AutoCAD-specific host facilities.

5.3.2 autolisp-host-bricscad

Responsibility:

- Adapter to BricsCAD-specific host facilities.

5.3.3 `autolisp-cli`

Responsibility:

- Standalone executable entry point.
- Batch evaluation.
- File loading.
- Test execution.
- REPL mode.

5.3.4 `autolisp-ide`

Responsibility:

- SLIME integration helpers.
- Inspector support.
- Breakpoint and stepping hooks.

6 Recommended Internal Representation

6.1 Reader Result

The reader should produce source-aware forms, not bare Common Lisp objects.

At minimum each parsed object should be able to retain:

- Source file.
- Line and column span.
- Original spelling where useful.
- Dialect metadata when relevant.

6.2 Symbol Model

Do not use raw Common Lisp package symbols as the semantic representation of AutoLISP symbols.

Instead define an AutoLISP symbol abstraction, or intern symbols in a dedicated runtime symbol table that is independent from Common Lisp packages.

Reason:

- Common Lisp package semantics are not AutoLISP semantics.
- This keeps case normalization and compatibility behavior under project control.
- It avoids package pollution and reader/package edge cases.

6.3 Environments

Represent environments explicitly as runtime data structures.

That is required to support:

- Dynamic scope.
- Introspection.
- Deterministic debugging.
- Future stepping and snapshot support.

7 Dialect Strategy

Dialect support should be explicit from the start.

Use a dialect descriptor object that controls:

- Reader differences.
- Builtin availability.
- Behavior flags.
- Feature predicates.
- Host capability advertisement.

Recommended initial dialects:

- `:autolisp-core`
- `:autocad`
- `:visual-lisp`
- `:bricscad`

Even if the first release implements mostly shared behavior, this structure should exist early so compatibility differences do not become scattered conditionals throughout the codebase.

8 Host Environment Profiles

CAD-target behavior and operating-system behavior should not be treated as the same thing.

The system should be able to run on one operating system while presenting another host-environment profile to AutoLISP / Visual LISP code.

Examples:

- Run on macOS while exposing a Windows-like AutoCAD host profile.
- Run on Linux while exposing a Windows-like BricsCAD host profile.
- Run on any supported OS while exposing a reduced pure-mock profile for tests.

This is especially important for:

- filesystem conventions,
- path syntax,
- platform capability predicates,
- secure loading behavior,
- host automation layers such as ActiveX / COM.

Accordingly, the architecture should include an explicit host-environment profile layer.

That layer should:

- describe the emulated target environment exposed to Lisp code,
- be selectable by command-line option or configuration,
- remain separate from the actual native execution platform,
- provide adapters or emulation for Windows-oriented facilities when a Windows-like profile is selected on non-Windows hosts.

This allows the project to target language and host compatibility directly, instead of hardwiring behavior to the native operating system.

9 Directory and System Layout

Target source tree shape:

```
sources/
  reader/
  runtime/
  builtins/
  conditions/
  host-api/
  testkit/
  vl/
  dcl/
  hosts/
  cli/
tests/
  reader/
  runtime/
  builtins/
  dialects/
  integration/
documentation/
  development-plan.org
  architecture-decisions.org
```

Target ASDF systems:

- clautolisp-base
- clautolisp/autolisp-reader

- `clautolisp-runtime`
- `clautolisp-vl`
- `clautolisp-dcl`
- `clautolisp-host-mock`
- `clautolisp-cli`
- `clautolisp-tests`

10 Execution Tracking

Actionable tasks for this subproject now live in ‘clautolisp/PLAN.md’.

This document remains the place for architecture, module boundaries, and implementation rationale.

11 Testing Strategy

11.1 Core Principles

- Every module must have implementation-independent tests that run on SBCL and CCL.
- Reader and evaluator tests are higher priority than UI or CAD integration tests.
- Prefer deterministic fixtures and golden files.
- Separate pure language tests from host-dependent tests.

11.2 Test Layers

- Reader tests should cover tokenization, parsing, source locations, encodings, line endings, and syntax diagnostics.
- Runtime tests should cover symbol lookup, dynamic scope, evaluation order, lambda application, special forms, and error propagation.
- Builtin conformance tests should cover function-by-function behavior, arity and type errors, and dialect-specific availability.

- Mock host integration tests should cover entity behavior, selection sets, and snapshot diffs.
- Real host compatibility tests should remain optional integration suites outside the normal fast test path.

11.3 Test Matrix

- SBCL plus unit tests.
- CCL plus unit tests.
- SBCL plus mock-host integration tests.
- CCL plus mock-host integration tests.

11.4 Test Assets Needed Early

- Small AutoLISP source corpus.
- Malformed reader corpus.
- Encoded input corpus with multiple external formats.
- Mixed line-ending corpus.
- Dialect compatibility corpus.
- Snapshot fixtures for mock CAD state.

12 Short Decision Summary

- Choose a dedicated AutoLISP reader/parser, not only Common Lisp reader macros.
- Keep AutoLISP symbols and environments separate from Common Lisp packages and bindings.
- Introduce dialect descriptors from day one.
- Build the project around a mock host and conformance tests.
- Treat SBCL and CCL as first-class targets throughout development.